

Secure, Reliable & Scalable Cloud Infrastructure Simulation for an AI Healthcare Platform

DevSecOps, Cloud Engineering, Infrastructure, Reliability & Operations Assessment

Product Owner: AI.Prof

Version: 2.0

Target Duration: 3–4 Days

Implementation Scope: Infrastructure and operations simulation

Application Development: Not Required

Real Cloud Deployment: Not Required

1. Project Overview

This project evaluates your ability to design, secure, automate, operate, monitor, troubleshoot, and recover a production-style cloud environment for an AI-powered healthcare platform.

You are **not required to build the healthcare application itself**. Instead, you will work around a simulated application consisting of an API, AI/agent service, background worker, database, queue, scheduling/integration workflows, and an external healthcare system.

Your responsibility is to design and implement the infrastructure required to run such a system reliably.

The project should demonstrate how you would answer questions such as:

- How should traffic enter the platform?
- Which services should be public or private?
- How should services communicate?
- How are secrets protected?
- How is infrastructure created reproducibly?
- How does code move safely from development to production?
- What happens when a service fails?
- How do operators detect failures?
- How does the system recover?

- How does the infrastructure scale?
- How can a failed deployment be rolled back?
- How can security failures prevent unsafe releases?

The environment is a **simulation/lab**, so a real cloud account is not required.

The emphasis is on engineering decisions and operational behavior rather than healthcare application features.

2. Project Philosophy & Scope

The core scenario is simple:

An application team tells you that an AI-powered healthcare platform needs to run securely, reliably, and at scale.

Your task is to design the infrastructure and operational environment that makes this possible.

The final system should behave like a small production cloud environment even though it may run entirely on a local machine or controlled simulation environment.

The implementation should reproduce important cloud engineering concepts including networking, compute, containers, infrastructure as code, deployment automation, security controls, monitoring, logging, alerting, failure recovery, and scaling.

The exact technologies are intentionally left open. You should select tools appropriate to the problem and explain your decisions.

The assessment is therefore not primarily about whether you know a particular cloud provider or framework. It is about whether you can reason about infrastructure as an engineer.

3. Simulated Healthcare Platform

The application layer is intentionally lightweight. You may use simple mock services rather than implementing real healthcare functionality.

The simulated platform should contain the following logical components.

Patient/API Service

The API represents the internet-facing entry point for the healthcare platform.

It should support controlled ingress, health checks, secure communication, and horizontal scaling. The API should not expose internal infrastructure directly.

AI/Agent Service

The AI service represents an internal service used by the application.

It should communicate through controlled internal paths and, where applicable, have controlled access to external AI services.

You do not need to implement an actual AI agent. A simple mock service is sufficient.

The infrastructure should nevertheless demonstrate appropriate secret handling, resource controls, service isolation, and monitoring.

Background Worker

The worker represents asynchronous processing such as appointment processing, notifications, EHR synchronization, and other background workflows.

The worker must generate measurable operational behavior.

At minimum, the environment should allow you to observe queue depth, processing latency, failed jobs, retry behavior, worker health, and worker restarts.

You must also be able to simulate worker failure and demonstrate recovery.

Database

A relational database should represent application state such as hospitals, doctors, patients, appointments, scheduling state, workflow state, and integration state.

The complete healthcare data model is not required.

The database must be treated as a private resource and must not be directly accessible from the public internet.

External EHR / Healthcare System

The EHR is represented by a mock external service.

It should support multiple failure conditions so that the infrastructure can be tested against dependency failures.

The simulation should be capable of representing successful requests, slow responses, timeouts, temporary failures, authentication failures, unavailable services, and situations where the outcome of an operation is unknown.

Queue / Background Processing

A queue or equivalent asynchronous mechanism should connect background workloads.

The environment must make it possible to simulate normal workload, increased workload, queue growth, worker failure, worker recovery, and retry behavior.

4. Infrastructure Objectives & Simulation Boundary

You own everything required to operate the simulated application.

This includes infrastructure, networking, security, containers or compute, deployment automation, monitoring, logging, alerting, reliability, scaling, and recovery.

You do **not** own the implementation of healthcare product features.

The final architecture should conceptually contain:

- A controlled public entry point.
- Public-facing API access.
- Private internal services.
- Background workers and queue processing.
- A private database.
- Controlled communication with external systems.
- Monitoring and security boundaries.

You should provide one clear architecture diagram in your documentation showing these boundaries.

Avoid creating multiple diagrams that communicate the same architecture. Additional sequence or state diagrams should only be included when they materially improve understanding.

5. Simulation Environment

The entire project must be runnable in a controlled environment.

You may use local containers, infrastructure simulators, local orchestration, simulated cloud resources, local networking, mock external services, or another equivalent approach.

The goal is to reproduce important cloud engineering behavior without requiring an actual production cloud environment.

The architecture should nevertheless be designed so that the conceptual transition to a real cloud environment would require minimal redesign.

Your submission should allow an evaluator to start the environment, inspect the infrastructure, generate workload, deploy changes, introduce failures, observe the resulting behavior, and recover the system.

6. Infrastructure as Code

Infrastructure must be represented as code.

Manual infrastructure creation must not be the primary mechanism.

Your infrastructure code should cover the major components required by the system, including networking, compute or containers, load balancing or ingress, security controls, database infrastructure, queues, monitoring, secrets/configuration, and environment configuration.

The infrastructure should be:

- Reproducible.
- Version controlled.
- Parameterized.
- Repeatable.
- Environment aware.
- Destroyable and recreatable.

Development and production-like environments should be represented without unnecessarily duplicating the entire infrastructure definition.

A reviewer should be able to understand what infrastructure exists, why it exists, and how it can be recreated.

7. Environment Management

At minimum, provide two logical environments:

Development and **Production-like**.

The environments should have appropriate separation while sharing reusable infrastructure definitions where possible.

Environment-specific values such as configuration, resource sizing, endpoints, credentials, or deployment settings should not require copying the entire infrastructure implementation.

The approach should demonstrate that the infrastructure could evolve into a multi-environment engineering workflow.

8. Network Architecture

The network architecture must clearly separate public and private resources.

Only components that genuinely require internet-facing access should be exposed publicly.

The public side should contain the controlled ingress or API entry point.

Private resources should include the database, workers, queues, internal services, and management interfaces wherever applicable.

Private resources should not receive unnecessary direct internet exposure.

Inbound and outbound communication should be intentionally controlled rather than allowing unrestricted connectivity.

Administrative access should also be restricted.

The architecture documentation must explain the major trust boundaries and why each resource is public or private.

9. Security Architecture

Security must be considered throughout the infrastructure rather than added at the end.

Network Security

The environment should restrict inbound traffic, minimize unnecessary exposed ports, isolate internal services, protect the database, and restrict administrative access.

Outbound communication should also be controlled where appropriate.

Identity and Access

Access should follow the principle of least privilege.

Application runtime, background workers, database access, deployment systems, monitoring systems, infrastructure administrators, and developers should not automatically receive unrestricted permissions.

Where separate identities or roles are appropriate, use them.

Secrets Management

Secrets must never be hard-coded into application source code, container images, infrastructure code, public configuration, or version control.

The same principle applies to database credentials, API credentials, service credentials, external integration credentials, and AI provider credentials.

Your design should explain where sensitive configuration is stored and how it reaches workloads securely.

10. Container & Runtime Security

Where containers are used, they should be designed with a security-first approach.

Images should be kept minimal and unnecessary packages should be avoided.

Services should run as non-root users where practical.

Dependencies should be validated for known vulnerabilities.

Container configuration should avoid unnecessary privileges and unsafe runtime settings.

The infrastructure should also reduce the impact of a compromised workload through appropriate isolation, permissions, network controls, and secret access.

The submission must include a mechanism for identifying container vulnerabilities.

11. Infrastructure Security Validation

Your infrastructure should be evaluated for common configuration risks.

The security process should be capable of identifying issues such as:

- Public exposure of private resources.
- Overly permissive access.
- Dangerous network rules.
- Hard-coded secrets.
- Insecure infrastructure configuration.
- Weak container configuration.
- Vulnerable dependencies or images.

You may choose the tools and techniques used to perform these checks.

The important requirement is that security validation produces actionable results and is integrated into the engineering workflow.

12. DevSecOps CI/CD Pipeline

The project must contain a simulated CI/CD pipeline.

The pipeline should perform the major stages required to safely move a change toward deployment.

A suitable workflow should include validation, testing, security checks, infrastructure validation, container or artifact creation, artifact validation, deployment, health verification, and successful completion or rollback.

Security must be integrated into the pipeline rather than treated as a final manual review.

Security checks may include source or dependency scanning, container image scanning, infrastructure security validation, secret detection, and configuration validation.

At least one meaningful security failure must be demonstrated.

The evaluator should be able to see that a failed security check can prevent an unsafe deployment from proceeding.

13. Deployment & Release Safety

Deployments must be versioned and controlled.

A new release should not immediately replace a healthy version without verification.

The system should provide a mechanism for checking readiness and health before traffic is shifted or the previous version is removed.

The exact deployment strategy is your choice.

A zero-downtime simulation should demonstrate the following behavior:

1. A new version is deployed.
2. The new version becomes ready.
3. Health is verified.
4. Traffic is shifted in a controlled manner.
5. The old version is removed only after the new version is considered healthy.

You must also demonstrate the opposite case.

If the new version fails health verification, the previous healthy version should remain available, rollout should stop, and the system should recover or roll back.

14. Reliability, High Availability & Resilience

The infrastructure must be designed to tolerate common operational failures.

At minimum, consider:

- API service failure.
- Worker failure.
- Container or instance failure.
- External EHR failure.
- Queue backlog.
- Database connectivity failure.
- Deployment failure.
- Configuration failure.

Critical services should avoid unnecessary single points of failure.

Appropriate mechanisms may include multiple instances, health checks, load distribution, automatic restart or replacement, failure isolation, retry mechanisms, and dependency-aware recovery.

Multi-region deployment is **not required**.

The assessment is more interested in whether you understand failure modes and can design sensible recovery behavior.

15. Scalability & Performance

The infrastructure should be capable of scaling without requiring a fundamental architectural redesign.

API capacity and background worker capacity should be independently scalable.

The simulation should demonstrate what happens when workload increases.

You should measure meaningful operational characteristics such as API latency, startup time, worker processing time, queue latency, database connectivity, network latency, resource utilization, and deployment duration where applicable.

Performance should be measured rather than simply assumed.

The documentation should identify potential bottlenecks and explain how the architecture would address them.

16. Observability

The environment must be observable enough for an operator to understand what is happening without manually inspecting every component.

Logging

Collect useful logs from application services, infrastructure, workers, deployments, and security-related events.

Logs should contain enough context to support troubleshooting.

Metrics

At minimum, monitor meaningful indicators such as request rate, error rate, latency, CPU, memory, queue depth, worker processing rate, worker restarts, deployment state, and service availability.

Health Checks

Services should expose appropriate health or readiness information.

Health checks should be useful for both deployment validation and operational monitoring.

Operational Dashboard

Provide an operational dashboard that makes the system state visible.

It should make it possible to determine whether the API is healthy, whether workers are processing work, whether the queue is growing, whether the database is reachable, whether deployments are healthy, whether external dependencies are failing, and whether resources are approaching saturation.

The exact dashboard technology is your choice.

17. Alerting

Alerts should be actionable rather than simply generating large amounts of noise.

At minimum, consider alerts for:

- API unavailability.
- High error rate.
- Excessive latency.
- Worker failure.
- Queue backlog.
- Database unavailability.
- Deployment failure.
- Resource exhaustion.
- Security validation failure.

The documentation should explain why the chosen alerts matter and what an operator should do when they fire.

18. Incident Simulation & Response

You must intentionally introduce at least **two meaningful infrastructure or operational failures**.

Examples include an API failure, worker backlog, external EHR outage, failed deployment, database connectivity failure, or another realistic infrastructure incident.

For each failure, demonstrate the operational lifecycle:

Failure → Detection → Investigation → Root Cause → Recovery → Verification → Prevention

The system should provide enough telemetry for the evaluator to understand how the failure was detected and investigated.

At least one incident must be documented in detail.

The incident report should describe what failed, the impact, how it was detected, the root cause, recovery steps, evidence supporting the investigation, and what preventive action should be taken.

19. Backup, Recovery & Disaster Considerations

Important application state should not be treated as disposable.

Your design should address database backup and recovery, infrastructure recreation, configuration recovery, and persistence of important state.

You should explain what happens if the database or another important stateful component is lost.

Full enterprise disaster recovery is not required.

However, the submission should demonstrate that you understand recovery objectives and the difference between recreating infrastructure and recovering persistent state.

20. Infrastructure Resilience & Failure Analysis

Identify potential single points of failure across the system.

This analysis should cover critical areas such as the API, workers, queue, database, external healthcare system, deployment system, and monitoring.

For each important dependency, explain what happens if it becomes unavailable.

The goal is not to eliminate every possible failure.

The goal is to demonstrate that you understand which failures matter, how they affect the system, and what trade-offs are reasonable for the simulated environment.

21. Operational Access & Auditability

Operational access must be controlled.

The system should distinguish, where appropriate, between developer access, deployment access, infrastructure administration, runtime permissions, and monitoring access.

Avoid giving every user or component unrestricted administrative privileges.

The infrastructure should also provide reasonable auditability.

A reviewer should be able to determine:

- Who initiated a deployment.
- Which version was deployed.
- What infrastructure changes occurred.
- Which security checks ran.
- Whether deployment succeeded or failed.
- When an incident occurred.
- What recovery action was taken.

The exact implementation can vary according to the selected tooling.

22. Cost Awareness

Even though no real cloud billing is required, the architecture should demonstrate cost awareness.

Identify resources that could become expensive, resources that should scale dynamically, resources that can remain minimal, and areas where overprovisioning could occur.

Your architecture decisions should acknowledge the trade-off between reliability, performance, security, and cost.

A production engineer should understand not only whether something works, but also what it costs to operate.

23. Required Simulation Scenarios

The final environment must support a consistent set of demonstrations.

Normal Operation

Start the infrastructure and demonstrate that the API, internal services, worker, queue, database, and external dependency operate normally.

Show relevant logs, metrics, health checks, and dashboard information.

Increased Workload

Generate additional API or background workload and demonstrate how the system behaves.

Show queue growth, worker processing, latency, resource usage, and scaling behavior where implemented.

Worker Failure

Stop or break a worker and demonstrate how the queue behaves, how the failure is detected, and how processing resumes after recovery.

External Dependency Failure

Make the simulated EHR slow, unavailable, or otherwise fail.

Demonstrate appropriate timeout, retry, failure handling, monitoring, and recovery behavior.

Deployment

Deploy a new healthy version and demonstrate controlled rollout and health verification.

Failed Deployment

Deploy a deliberately unhealthy version.

The system should detect the problem and prevent or reverse the unsafe release while preserving service availability where possible.

Infrastructure Recovery

Destroy or restart an important infrastructure component and demonstrate how the environment recovers or can be recreated.

These scenarios should be repeatable enough for an evaluator to verify the implementation.

24. Technology Selection

No specific infrastructure, cloud, orchestration, CI/CD, security, monitoring, or simulation technology is mandated.

You are free to select tools appropriate to the problem.

Your choices may include technologies for:

- Infrastructure as code.
- Containers and runtime.
- Local cloud simulation.
- Networking.
- CI/CD.
- Security scanning.
- Monitoring.
- Logging.
- Metrics.
- Alerting.
- Deployment.
- Configuration.
- Secrets management.

The important requirement is not the number of tools used.

You must be able to explain **why** each significant technology was selected, what problem it solves, and what trade-offs it introduces.

Using a large number of tools without understanding them will not be considered a strong submission.

25. Documentation & Deliverables

Your final submission should contain the following.

1. Architecture Document

Document the overall system architecture, network boundaries, security model, deployment approach, reliability strategy, observability approach, failure scenarios, recovery mechanisms, and major technology decisions.

2. Infrastructure as Code Repository

Provide all infrastructure definitions required to recreate the environment.

3. Simulation Environment

Provide the simulated application services, mock dependencies, infrastructure configuration, workload generation, and failure mechanisms required to demonstrate the system.

4. CI/CD Pipeline

Provide the complete pipeline configuration, including validation, testing, security checks, build, deployment, health verification, and rollback behavior.

5. Security Report

Document the security checks performed, findings discovered, severity or impact, remediation, and evidence that the relevant controls are working.

6. Incident Report

Document at least one meaningful incident from detection through recovery and prevention.

7. Demonstration

Provide a walkthrough demonstrating architecture, infrastructure, network isolation, security controls, CI/CD, deployment, monitoring, failure simulation, recovery, rollback, and major engineering trade-offs.

A single primary architecture diagram is required. Additional diagrams should only be included when they communicate information that cannot be explained clearly through the architecture document.

26. Explicitly Not Required

The following are outside the scope of the project:

- Healthcare frontend.
- Patient UI.
- Hospital dashboard.
- Doctor dashboard.
- Full AI agent implementation.
- Voice assistant.
- Telephony.
- Real EHR integration.
- Real patient database.

- Production healthcare application.
- Real patient data.
- Real cloud deployment.

All healthcare application behavior and external dependencies may be simulated.

The candidate is being evaluated on infrastructure and operational engineering rather than product development.

27. Simulation vs. Implementation Boundary

The simulated application exists only to provide realistic infrastructure behavior.

The candidate owns the infrastructure around the application runtime.

This includes:

Network → Security → Compute/Containers → Deployment → Monitoring → Reliability → Recovery → Operations

The simulated application provides the workloads and dependencies necessary to test those capabilities.

This separation is intentional.

You should spend your implementation effort on infrastructure engineering rather than building sophisticated business functionality.

28. Expected Engineering Thinking

Throughout the project, your decisions should answer questions such as:

Architecture: Why is the database private? Why is the API public? Where are the trust boundaries? How do internal services communicate?

Security: What happens if a container is compromised? How are credentials protected? What permissions does each workload actually require? How are insecure infrastructure configurations detected?

Reliability: What happens when the API dies? What happens when a worker stops? What happens when the queue grows? What happens when the EHR becomes unavailable?

Deployment: How do you know a release is healthy? What happens when it is not? How do you prevent an unhealthy version from replacing a healthy one?

Observability: How does an operator know something is failing? How do they determine the affected component? What evidence helps identify the root cause?

Scalability: Which components scale independently? What happens when workload increases? Where are the likely bottlenecks?

The strongest submissions demonstrate these decisions through both implementation and explanation.

29. Strong Submission Characteristics

A strong submission will demonstrate clear public/private boundaries, reproducible infrastructure, appropriate least-privilege access, protected secrets, meaningful security validation, secure containerization, automated deployment, health checks, controlled releases, rollback, useful monitoring, actionable alerts, failure simulation, recovery, and strong documentation.

It should also explain trade-offs rather than simply presenting infrastructure files.

The evaluator should be able to understand not only **what** was built, but **why** it was designed that way.

30. Weak Submission Characteristics

Weak submissions typically expose too many services publicly, make the database directly accessible, hard-code secrets, rely heavily on manual infrastructure setup, lack reproducible IaC, perform security checks only manually, provide no rollback strategy, have weak health checks, lack useful monitoring, cannot simulate meaningful failures, provide no recovery behavior, or grant unrestricted administrative access.

Infrastructure that works only on the candidate's machine but cannot be recreated reliably is also considered weak.

Likewise, simply configuring tools without demonstrating understanding of their purpose will not score highly.

31. AI-Assisted Engineering

AI tools are permitted and may be used during the project.

However, the candidate remains responsible for the final architecture, infrastructure correctness, security configuration, testing, failure analysis, deployment behavior, and operational decisions.

AI-generated infrastructure must be reviewed and validated by the candidate.

You should document meaningful AI assistance, particularly where AI materially contributed to architecture exploration, implementation, troubleshooting, or documentation.

The purpose of allowing AI is not to eliminate engineering judgment.

The assessment is intended to evaluate whether you can effectively use AI while still understanding and owning the resulting system.

32. Final Demonstration

The final demonstration should tell a coherent operational story.

Start the environment and show normal operation.

Demonstrate the architecture and network boundaries.

Show the infrastructure and how it is recreated.

Run the CI/CD pipeline and demonstrate security validation.

Deploy a healthy version and show health verification.

Introduce a meaningful failure and demonstrate detection, alerting, investigation, and recovery.

Then deploy an intentionally broken version and demonstrate how health validation prevents or reverses the unsafe release.

Finally, explain the major security, reliability, scalability, cost, and architectural trade-offs.

The evaluator may ask you to modify or troubleshoot the system during the demonstration.

You should therefore understand the implementation rather than simply relying on pre-recorded output.

33. Definition of Done

Area	Completion Requirement
Infrastructure	Infrastructure is defined as code, reproducible, and re creatable
Networking	Public/private boundaries are implemented and unnecessary exposure is prevented
Security	No hard-coded secrets; access is restricted; security validation is automated
Containers	Images and runtime configuration follow reasonable security practices
CI/CD	Pipeline performs validation, security checks, deployment, health verification, and failure handling
Deployment	Releases are versioned and unhealthy deployments can be stopped or rolled back
Reliability	Service failure, worker failure, queue backlog, and external dependency failures can be demonstrated
Observability	Logs, metrics, health checks, dashboards, and actionable alerts are available
Recovery	Important infrastructure and persistent state have a documented recovery approach
Auditability	Deployment and operational actions can be traced sufficiently
Documentation	Architecture, security, reliability, incident response, and technology decisions are documented
Demonstration	Normal operation and meaningful failure/recovery scenarios can be reproduced

34. Final Project Definition

The final project is:

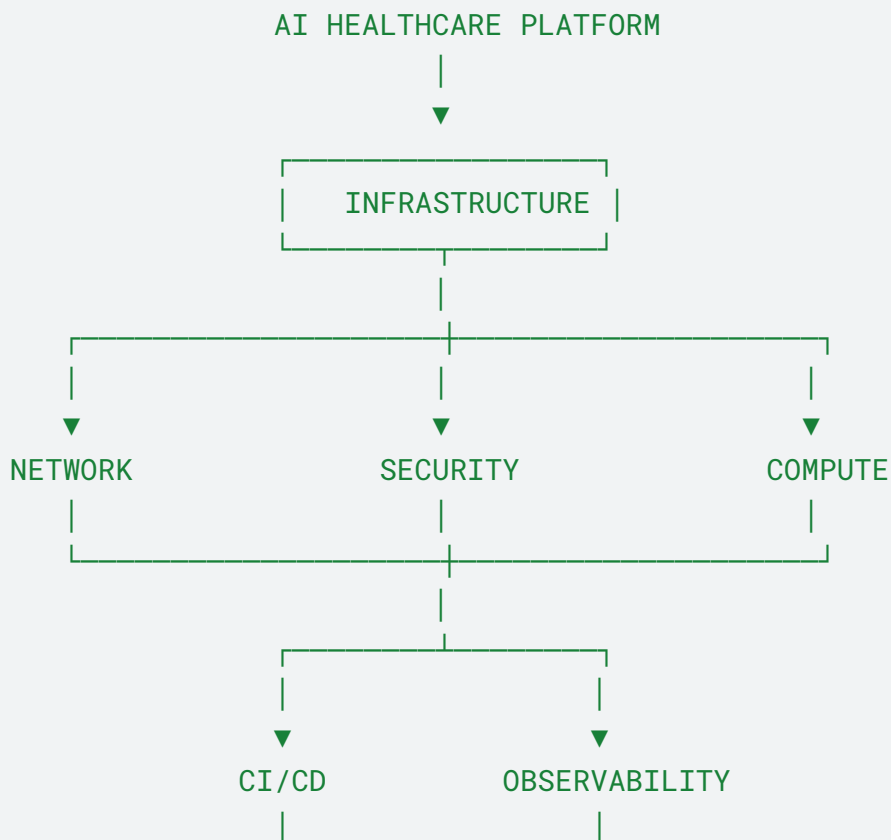
A secure, reliable, observable and scalable cloud infrastructure simulation for a modern AI-powered healthcare platform.

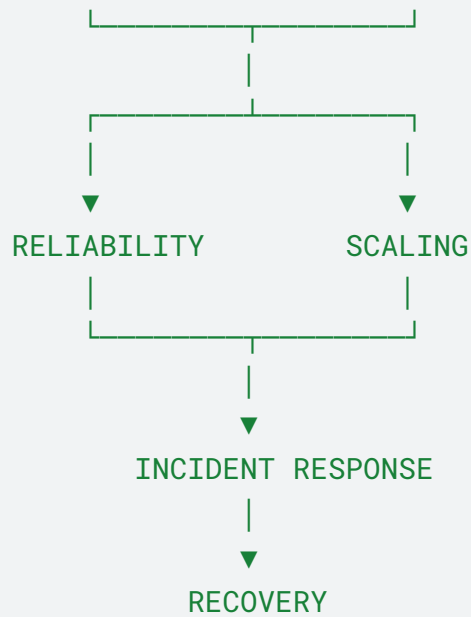
The candidate is **not building the healthcare product**.

The candidate is demonstrating how they would operate the infrastructure behind it.

The candidate should be able to show:

None





The central assessment question is:

Can the candidate design, secure, automate, observe, troubleshoot and recover a production-style cloud environment—even when the actual application is provided only as a simulation?

That is the core of this internship project. A successful submission should make the answer clearly **yes**.